

Лабораторная работа №4

Вычисление наибольшего общего делителя

Кюнкриков Даниил Саналович (НПИМд-01-24, 1132249574)

Содержание

1	Введение	2
2	Цель и задачи	2
2.1	Цель работы	2
2.2	Задачи	2
3	Теоретические сведения	2
3.1	Наибольший общий делитель	2
3.2	Алгоритм Евклида	2
3.3	Бинарный алгоритм Евклида (Стейна)	3
3.4	Расширенный алгоритм Евклида	3
3.5	Расширенный бинарный алгоритм Евклида	3
4	Выполнение лабораторной работы	3
4.0.1	Листинг 1 — Алгоритм Евклида	3
4.0.2	Листинг 2 — Бинарный алгоритм Евклида (Стейна)	4
4.0.3	Листинг 3 — Расширенный алгоритм Евклида	4
4.0.4	Листинг 4 — Расширенный бинарный алгоритм Евклида	5
5	Результаты	6
6	Выводы	7

Список Иллюстраций

1	Результат работы программы	6
-------------------	--	---

Список Таблиц

1 Введение

В данной лабораторной работе реализуются алгоритмы вычисления наибольшего общего делителя (НОД) для двух целых чисел: алгоритм Евклида, бинарный алгоритм Евклида (Стейна), расширенный алгоритм Евклида и расширенный бинарный алгоритм Евклида. Дополнительно для расширенных алгоритмов вычисляются коэффициенты Безу x и y , удовлетворяющие равенству $ax + by = \gcd(a, b)$.

2 Цель и задачи

2.1 Цель работы

Реализовать на языке Python несколько алгоритмов вычисления НОД и продемонстрировать их работу на примерах.

2.2 Задачи

1. Реализовать алгоритм Евклида.
2. Реализовать бинарный алгоритм Евклида (Стейна).
3. Реализовать расширенный алгоритм Евклида (получение коэффициентов Безу).
4. Реализовать расширенный бинарный алгоритм Евклида (получение коэффициентов Безу).
5. Провести тестирование на нескольких наборах входных данных и сравнить результаты с `math.gcd`.

3 Теоретические сведения

3.1 Наибольший общий делитель

НОД двух целых чисел a и b — это наибольшее целое число d , делящее оба числа без остатка.

Ключевое свойство (лемма Евклида): если $a = qb + r$, то

$$\gcd(a, b) = \gcd(b, r).$$

Отсюда следует классический алгоритм Евклида: заменять пару (a, b) на $(b, a \bmod b)$ до тех пор, пока остаток не станет равен нулю.

3.2 Алгоритм Евклида

На каждом шаге вычисляется остаток $r = a \bmod b$, затем выполняется переход $(a, b) \leftarrow (b, r)$. Последний ненулевой остаток и есть НОД.

3.3 Бинарный алгоритм Евклида (Стейна)

Алгоритм использует свойства:

- если a и b чётные, то $\gcd(a, b) = 2 \cdot \gcd(a/2, b/2)$;
- если a чётное, b нечётное, то $\gcd(a, b) = \gcd(a/2, b)$;
- если a и b нечётные, то $\gcd(a, b) = \gcd(|a - b|, \min(a, b))$.

Преимущество — замена деления на операции деления на 2 (битовые сдвиги) и вычитания.

3.4 Расширенный алгоритм Евклида

Помимо $d = \gcd(a, b)$ алгоритм находит x и y , такие что:

$$ax + by = d.$$

Коэффициенты вычисляются вместе с остатками, поддерживая инвариант представления текущего остатка как линейной комбинации a и b .

3.5 Расширенный бинарный алгоритм Евклида

Это расширение бинарного алгоритма Евклида, где дополнительно поддерживаются коэффициенты линейных комбинаций для текущих значений (инварианты вида $u = Aa + Bb$, $v = Ca + Db$). При делении на 2 выполняются корректировки коэффициентов, чтобы сохранялась целочисленность.

4 Выполнение лабораторной работы

Программная реализация алгоритма Евклида показана на [Листинге №1](#).

4.0.1 Листинг 1 — Алгоритм Евклида

```
def euclidean_gcd(a,b):
    r_prev, r_curr = a,b

    while r_curr !=0:
        r_next = r_prev % r_curr
        r_prev = r_curr
        r_curr = r_next

    d = r_prev
    return d

vala = 12345
valb = [24690, 54321, 12541]
for val in valb:
    print(f"GCD({vala}, {val}) = {euclidean_gcd(vala, val)}")
```

Программная реализация бинарного алгоритма Евклида показана на [Листинге №2](#).

4.0.2 Листинг 2 — Бинарный алгоритм Евклида (Стейна)

```
def binareuc_gcd(a,b):
    g = 1

    while a%2 == 0 and b%2 ==0:
        a //=2
        b //= 2
        g *= 2

    u, v = a, b

    while u != 0:
        while u% 2 ==0:
            u //= 2
        while v% 2 ==0:
            v //= 2
        if u >= v:
            u = u-v
        else:
            v = v-u

    d = g * v

    return d

vala = 12345
valb = [24690, 54321, 12541]
for val in valb:
    print(f"GCD({vala}, {val}) = {binareuc_gcd(vala, val)}")
```

Программная реализация расширенного алгоритма Евклида показана на [Листинге №3](#).

4.0.3 Листинг 3 — Расширенный алгоритм Евклида

```
def extendedeuc_gcd(a,b):
    r_prev, r_curr = a,b
    x_prev, x_curr = 1, 0
    y_prev, y_curr = 0, 1

    while r_curr !=0:
        q = r_prev // r_curr
        r_next = r_prev % r_curr
        x_next = x_prev - q * x_curr
        y_next = y_prev - q * y_curr
```

```

    y_next = y_prev - q * y_curr

    r_prev, r_curr = r_curr, r_next
    x_prev, x_curr = x_curr, x_next
    y_prev, y_curr = y_curr, y_next

    d, x, y = r_prev, x_prev, y_prev
    return d, x, y_curr

d1, x1, y1 = extendedeuc_gcd(105,91)
d2, x2, y2 = extendedeuc_gcd(154, d1)
print(f"GDC(105,91) = {d1, x1, y1}")
print(f"final GDC(154,7) = {d2, x2, y2}")

```

Программная реализация расширенного бинарного алгоритма Евклида показана на [Листинге №4](#).

4.0.4 Листинг 4 — Расширенный бинарный алгоритм Евклида

```

def extendbineuc_gcd(a,b):
    g = 1
    while a%2 ==0 and b%2 == 0:
        a //=2
        b //= 2
        g *= 2

    u, v = a, b
    A, B = 1, 0
    C, D = 0, 1

    while u != 0:
        while u % 2 ==0:
            u //=2
            if A % 2 == 0 and B % 2 == 0:
                A //= 2
                B //= 2
            else:
                A = (A + b) // 2
                B = (B - a) // 2
            print("v: " + str(v))

        while v%2==0:
            v //= 2
            if C % 2 == 0 and D % 2 == 0:
                C //= 2
                D //= 2
            else:

```

```

        C = (C+b) // 2
        D = (D-a) // 2
    print("u: " + str(u))

    if u >= v:
        u = u - v
        A = A - C
        B = B - D
    else:
        v = v - u
        C = C - A
        D = D - B
d = g * v
x = C
y = D
return d, x, y

```

```

dbin,xbin,ybin = extendbineuc_gcd(105, 91)
print(f"GDC(105,91) = {dbin,xbin,ybin}")

```

5 Результаты

Для демонстрации работы алгоритмов использовались следующие пары чисел: (12345, 24690), (12345, 54321), (12345, 12541), а также пример для коэффициентов Безу (105, 91).

Результат запуска программы изображен на Рисунок 1.

```

GCD(12345, 24690) = 12345
GCD(12345, 54321) = 3
GCD(12345, 12541) = 1
GCD(12345, 24690) = 12345
GCD(12345, 54321) = 3
GCD(12345, 12541) = 1
GDC(105,91) = (7, -6, -15)
final GDC(154,7) = (7, 0, -22)
v: 91
u: 7
u: 7
u: 7
GDC(105,91) = (7, -6, 7)

```

Рисунок 1: Результат работы программы

Ожидается, что значения НОД, полученные разными реализациями, совпадут с `math.gcd`, а проверка линейной комбинации даст $ax + by = d$.

6 Выводы

В ходе выполнения лабораторной работы были реализованы четыре алгоритма вычисления НОД: Евклида, бинарный Евклида, расширенный Евклида и расширенный бинарный Евклида. Для расширенных алгоритмов получены коэффициенты Безу и показана корректность результата через проверку $ax + by = d$.